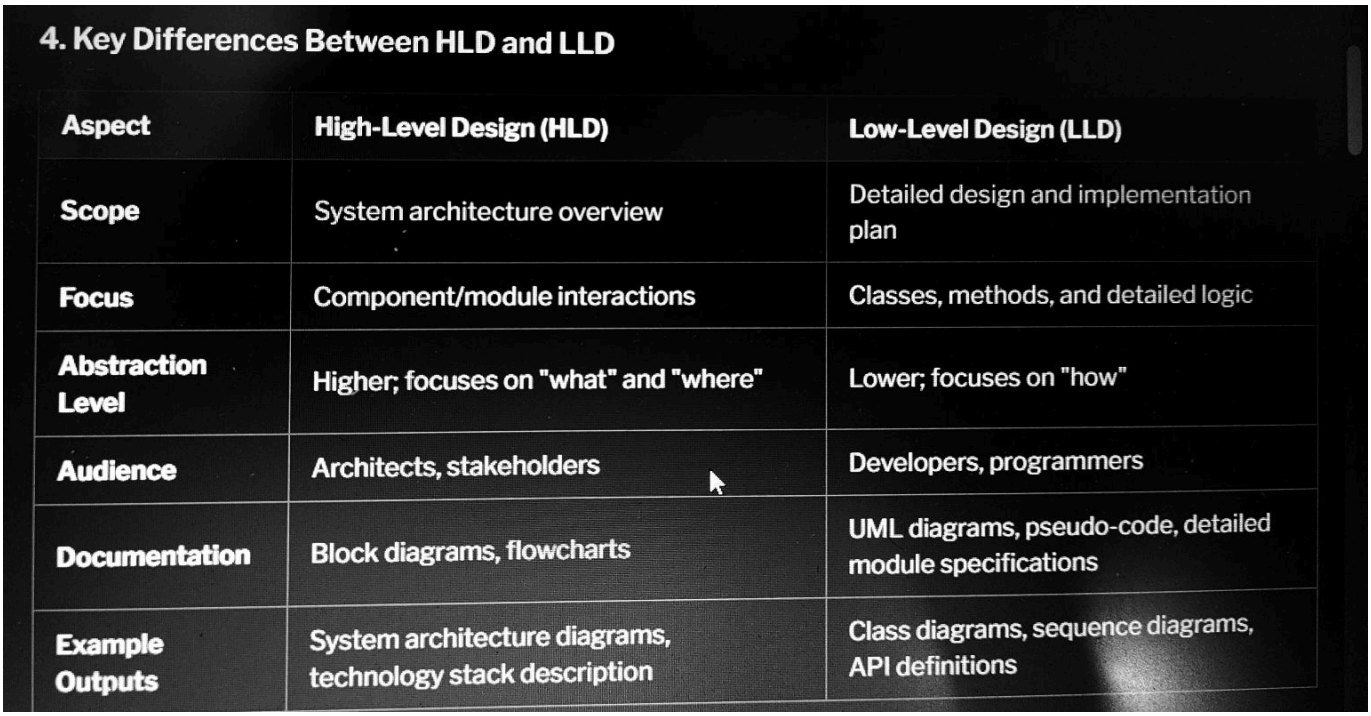
Hld:- for macro level architecture design entire system, while lld:- for micro level components how each modules classes, methods and data interactions works.



4. Key Differences Between HLD and LLD

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Scope	System architecture overview	Detailed design and implementation plan
Focus	Component/module interactions	Classes, methods, and detailed logic
Abstraction Level	Higher; focuses on "what" and "where"	Lower; focuses on "how"
Audience	Architects, stakeholders	Developers, programmers
Documentation	Block diagrams, flowcharts	UML diagrams, pseudo-code, detailed module specifications
Example Outputs	System architecture diagrams, technology stack description	Class diagrams, sequence diagrams, API definitions

Design patterns are the foundation of good software design. They help you solve recurring problems and improve the structure and quality of your code.

Create a card class
And inherite for credit card and debit card
Reduce code duplication

-> In typescript we don't need let and const to create class properties this is for variables
-> Abstract method no body ther child class have to implement it.
U can't create object of class directly with new for abstract class
-> To run small ts file also we need nodemoudle and tsconfig file

`node_modules` — Why needed?

TypeScript is **not built into Node.js**. So you need to **install** it as a tool:

- `typescript` → the compiler (`tsc`) that converts `.ts` → `.js`
- `ts-node` → runs `.ts` directly without manual compile step
- `@types/node` → tells TypeScript what Node.js functions like `console.log` look like

All installed packages live in `node_modules/`.

---

tsconfig.json — Why needed?

It tells the TypeScript compiler **how to behave**:

Setting	Meaning
`"module": "commonjs"`	How to handle `import/export` (Node.js style)
`"target": "es6"`	Which JavaScript version to compile to
`"outDir": "./dist"`	Where to put the compiled `.js` files
`"strict": true`	Enable strict type checking

Without it, `tsc` uses defaults which may not work for your project.

Simple Summary

```
You write  →  .ts file
tsc reads  →  tsconfig.json (for rules)
tsc uses   →  node_modules/typescript (the tool)
tsc outputs → .js file  →  Node.js runs it ✅
```

Think of it like:

- `node_modules` = **tools in your toolbox**
- `tsconfig.json` = **instruction manual** for those tools

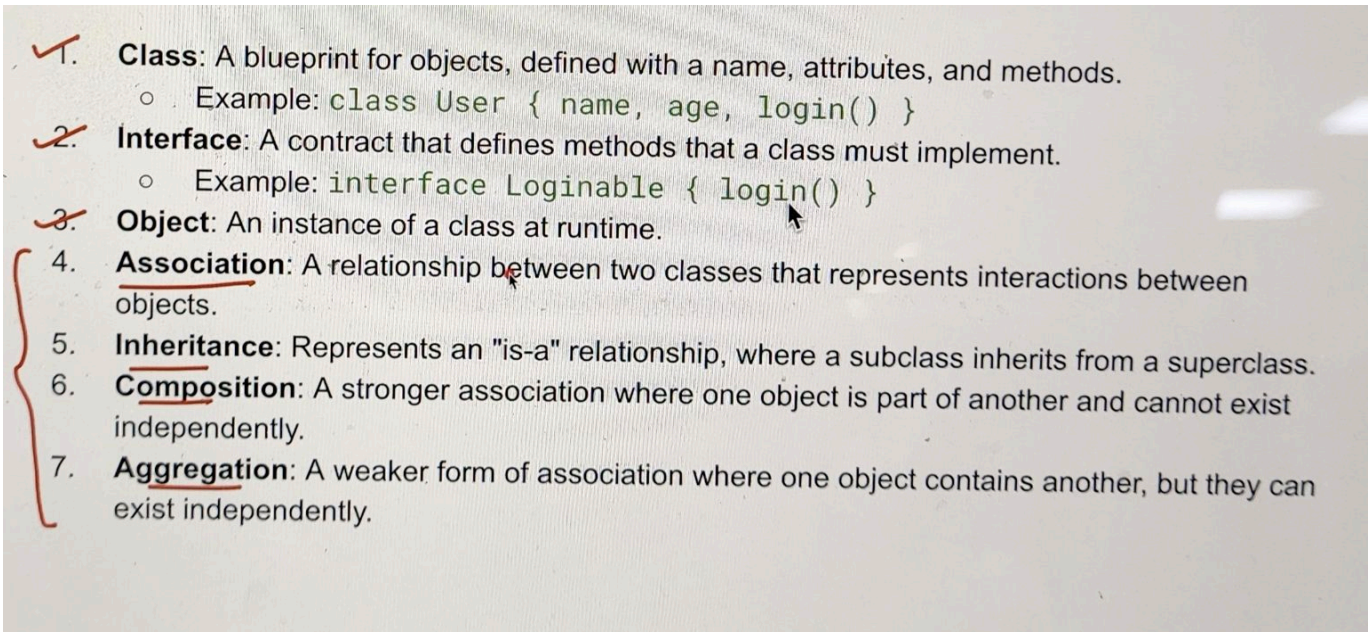
Uml:-Unified Modeling Language to model systems

> The idea is to have a uniform way to represent the classes, objects, relationships and interactions within simple or complex systems to make it easier for developers and stakeholders to understand and communicate about the system.

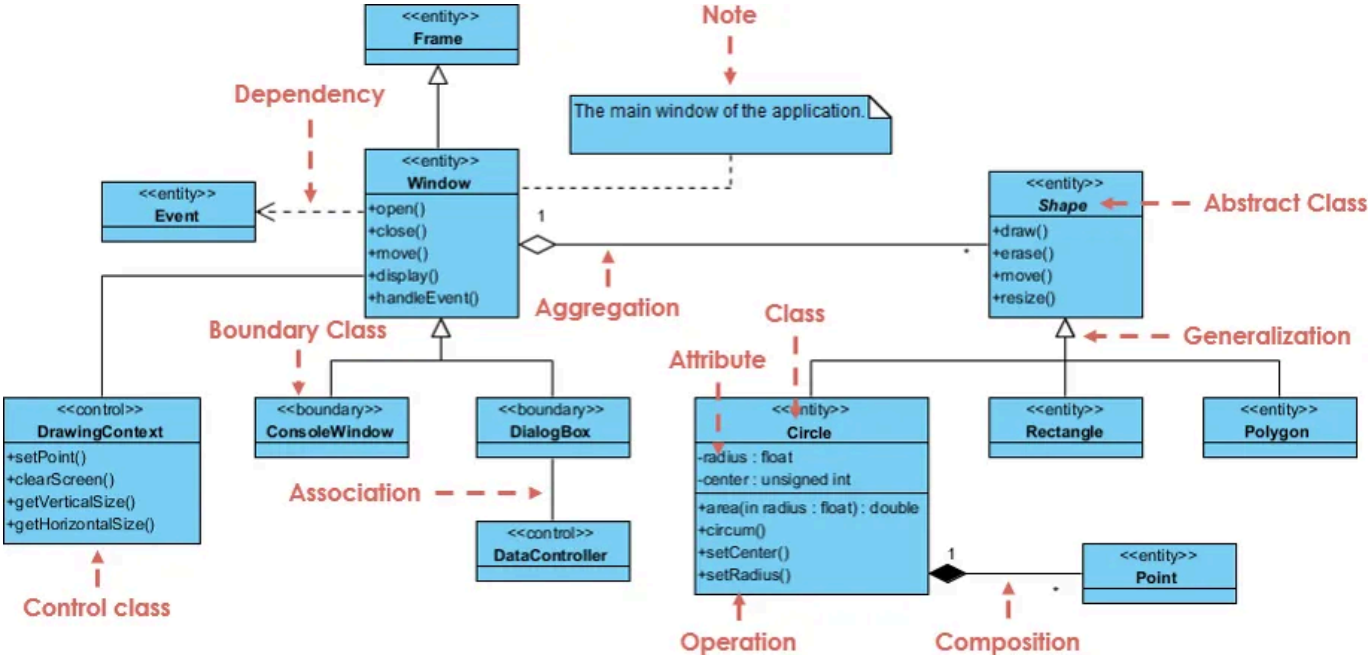
Reason of uml diagram

Visualization: UML diagrams provide a visual representation of a system, making it easier to understand the structure, relationships, and interactions between components.
Documentation: They serve as detailed documentation for the software architecture, which is useful for maintaining and scaling the system.
Communication: UML diagrams are a common language for software designers, developers, and stakeholders, facilitating communication about the design.
Standardization: Since UML is a standardized language, it ensures that everyone involved understands the system's design in the same way.

Basic Elements



Class Diagram: Represents the static structure of a system, showing classes, attributes, methods, and the relationships between them (inheritance, association, etc.).



Class Notation (The Box): A three-compartment rectangle representing an entity:

Visibility Modifiers: Special symbols indicating access control for fields and functions:

+ Public: Accessible anywhere in the system.

- Private: Accessible only within that specific class.

 # Protected: Accessible within the class and its subclasses.

~ Package Private: Accessible within the same package directory.

Class Relationships & Notations

Relationships define how your system components interact with each other

Relationship Type

(Meaning / Conceptual Rule)

Visual Line Style & Symbol

Inheritance / Generalization "IS-A" relation (where Subclass inheriting properties and behaviour from parent class).
Solid line with a hollow triangle pointing to parent.
Ex:- dog class inherit from animal class.

Realization / Implementation A class implementing an interface's blueprint (A class implements the behavior defined by an interface).
Dashed line with a hollow triangle pointing to interface.

Association "HAS-A" relation (represents a relationship between two or more classes. In this case, each object in one class is associated with one or more objects of another class).
Solid straight line (Can include an arrow for direction).
Ex:- one teacher class can teach many student class

Aggregation Weak form of association relation (is a weak "has-a" relationship where one class contains objects of another class. However, the contained objects can exist independently of the container object).
Solid line with a hollow diamond next to parent.
Ex:- in department many professors can exist and if there is no department still professors can exists, proof can exsit independently

Composition Strong "has-a" relation (is a strong "has-a" relationship, where one class owns objects of another class. If the container object is destroyed, the contained objects are destroyed as well).
Solid line with a filled diamond next to parent.
Ex:- room can't exist independently without house, if house destroy room should also destroy.

Dependency Temporary use (This is a relationship where one class relies on another in some way, often through method parameters, return types or temporary associations).
Dashed line with an open arrow.

Ex:- printer class need document class object in print method parameter
Printer class depend on document class.

The **SOLID principles** are five fundamental object-oriented design principles used to create software that is easy to maintain, scale, and understand by following a structured approch.
Shows how your classes are structured, broken down, and linked.

Types

1. **Single Responsibility Principle** (SRP)"A class should have one, and only one, reason to change." Class should not have more than one responsibilty to change. It should have one responsibility
: A User class should only handle user-related logic, while database-related operations should be handled by a separate UserRepository class.
Diff Work should be separated in diffrent class.
Uml repersentation:- three separate, small class blocks

```
Bad Approach: A single Invoice class that calculates totals, formats HTML, and saves data to a MySQL database.
Good Approach: Split the class into three distinct pieces: Invoice (data logic), InvoicePrinter (presentation), and InvoiceRepository (database storage).
```

2. **Open/Closed Principle** (OCP)"Software entities (classes, modules, functions) should be open for extension, but closed for modification."
: Adding new functionality to a system using inheritance or composition without modifying existing code.
We should not touch(disturb) pre built tested code we should extend not modify it, may be modification can cause bug and error. Modify when it's extremely important.
Uml representation :- dashed lines with hollow triangles (Realization).

```
Bad Approach: Using a giant switch or if/else statement inside a PaymentProcessor class to check if a payment method is CreditCard or PayPal. Adding a new payment type requires modifying this existing code.
```

Good Approach: Introduce a common PaymentMethod interface/abstract. Create separate classes for CreditCardPayment and PayPalPayment that implement this interface.

3. **Liskov Substitution Principle** (LSP)"states that Objects of a superclass should be replaceable with objects of its subclasses without breaking the application or altering the correctness of the program.
(Removing functionally or changing behaviour in a subclass violates this principle).
It ensures that a subclass can stand in for it's parent class and function correctly in any context that expects the parent class "

Objects of a superclass should be replaceable with objects of its subclasses without breaking the application.

UML Representation: Use solid lines with hollow triangles (Inheritance).

Bad Approach: Creating a `Bird` base class with a `fly()` method, and then creating a `Ostrich` subclass. Because an ostrich cannot fly, calling `fly()` will crash your system.

Good Approach: Split the hierarchy. Create a generic `Bird` class, and then introduce a specific `FlyingBird` subclass that contains the `fly()` method.

4. **Interface Segregation Principle** (ISP)"
The isp ensures that classes are not burdened with methods they don't need. It promotes better design by breaking large. Genral purpose interface into smaller, more specific ones.
It will improve maintainability, flexibility, and testability by ensuring that classes only have the dependencies they actually require.
(No client should be forced to depend on methods it does not use. This encourages splitting large, board interface into smaller, more specific onces.)
UML Representation: Instead of one massive interface block, draw multiple smaller interface blocks.

Bad Approach: A single Worker interface with work() and eatLunch(). If you create a Robot class implementing this interface, it is forced to implement eatLunch(), which makes no sense.

```
Good Approach: Break the interface into two small, targeted ones: Workable (with work()) and Feedable (with eatLunch()). A human class implements both, while a robot class implements only Workable.
```

5. **Dependency Inversion Principle** (DIP)
:- high level modules should not depend on low-level modules, both should depend on **abstractions**(interface).
"Depend upon abstractions, not concretions."
(Reduce the coupling between components) Dependency injection is a common way to achieve dip, but it is not mandatory.
UML Representation: - both blocks will draw arrows pointing toward a central Interface block.

```
Srp:- a class should have only one reason to change.should have only one responsibility or job
Ocp:- You should be able to add new functionality(extension) to a system without altering existing, working source code.
Lsp:-  objects of a superclass shall be replaceable with objects of its subclass without affecting the correctness of the program.
Isp:- Avoid creating massive, "fat" interfaces that force implementing classes to write boilerplate code for functions they do not need.
Dip:- High-level business logic modules should not rely directly on low-level implementation details (like databases or third-party APIs). Both should rely on shared abstractions (interfaces).
```

What are design patterns

Design patterns are reusable, time-tested solutions to common problems that occur during software design.
Patterns are blueprints that can be applied in multiple situations to solve challenges, but they are not direct pieces of code.
They focus on object interactions, system architecture, and how classes/objects communicate

The **Three Main CategoriesDesign patterns** are broadly divided into three distinct buckets, depending on the exact problem they solve:

Creational Patterns: Focus on how objects are created. They hide the creation logic instead of letting you instantiate objects directly using the new keyword.

Structural Patterns: Focus on how classes and objects compose to form larger structures. They use inheritance and interfaces to make different independent parts work together seamlessly.

Behavioral Patterns: Focus on communication between objects. They define how responsibilities, data, and algorithms flow across different blocks in your system.

Benefits of Design Patterns

Code Reusability: Patterns provide proven solutions that can be applied in multiple projects.

Maintainability: Encourages clean, understandable, and structured code, which is easier to maintain and extend

Communication: Design patterns act as a common language between developers, improving communication and understanding.

Scalability: Design patterns help in structuring code to support future scaling with fewer refactors.

Efficiency: Avoids reinventing the wheel by using well-tested and widely accepted solutions.